



NETRA-Mail Complete Project Handbook

Explainable multilingual phishing detection, SOC investigation and digital forensics

171	97.35%	0.042%	6
tests passed	URL holdout accuracy	URL false-positive rate	language profiles

Verified revision 0f300a0
16 September 2026

Prepared as a basic-to-advanced learning guide, presentation reference and repository map.

Contents

How to use this handbook	5
The 30-second explanation	5
The one-line pitch	5
1. The problem NETRA-Mail solves	6
Who uses it?	6
Why multilingual analysis matters	6
2. Email security from the beginning	6
2.1 What an email contains	6
2.2 What phishing means	7
2.3 The difference between identity and intent	7
3. SPF, DKIM, DMARC and ARC in plain language	7
3.1 SPF — who is allowed to send for the envelope domain?	7
3.2 DKIM — was the signed content changed?	7
3.3 DMARC — does authentication align with the visible From domain?	8
3.4 ARC — what happened through forwarding intermediaries?	8
3.5 How NETRA-Mail treats unavailable evidence	8
4. End-to-end system workflow	8
Step 1 — user chooses an email	8
Step 2 — backend validates the request	8
Step 3 — parsing and normalization	8
Step 4 — independent evidence engines run	8
Step 5 — evidence is fused	9
Step 6 — results are connected to investigation	9
5. Understanding the score and decision	9
5.1 Score, classification and confidence are different	9
5.2 Why evidence correlation matters	9
5.3 The IRCTC false-positive lesson	9
6. Detection engines and their contribution	9
6.1 Sender and header analysis	9
6.2 Multilingual and NLP analysis	9
6.3 URL analysis	9
6.4 Domain, IP and reputation intelligence	10
6.5 Attachments, archives, QR and OCR	10
6.6 Email ML and URL ML	10
6.7 Risk fusion	10
7. Investigation, forensics and reporting	10

7.1 SOC dashboard tabs	10
7.2 Cases and campaigns	10
7.3 Evidence integrity	10
7.4 Reports	10
8. Security, privacy and deployment	11
8.1 Main controls	11
8.2 Local and hosted operation	11
8.3 Why persistent storage matters	11
9. Metrics: the concepts you must understand	11
9.1 Confusion matrix	11
9.2 Metric formulas	11
9.3 Why the dataset split matters	12
10. Verified project metrics	12
10.1 Current automated regression suite	12
10.2 Domain-separated URL-model holdout	12
10.3 Seeded 25-email functional benchmark	13
10.4 Honest interpretation	13
11. Testing strategy	13
12. What NETRA-Mail does well	13
13. Current limitations and future work	13
Present limitations	13
Recommended roadmap	14
14. How to present NETRA-Mail to judges	14
A clear five-minute flow	14
Suggested closing statement	14
15. Common viva questions and answers	14
Is NETRA-Mail just an ML classifier?	14
How do you analyze a website not present in the dataset?	14
Why can a real phishing message pass SPF and DKIM?	15
Why not show an attacker location for every email?	15
What reduced the IRCTC false positive?	15
What is the most defensible metric?	15
16. Glossary	15
17. Repository architecture	16
18. File-by-file contribution reference	16
Repository root	17

backend	18
dashboard	20
extension	20
inspection	20
data	21
evaluation	22
tests	24
testingforai	26
scripts	27
docs	27
.github	28
.streamlit	28
evaluation_results	28
 19. Final presenter checklist	 28
 20. Source references inside the repository	 28

How to use this handbook

This document teaches NETRA-Mail from first principles and then moves into implementation detail. Read Chapters 1–5 before presenting the idea. Use Chapters 6–13 for a technical review, Chapters 14–17 for metrics and limitations, and the appendices for viva questions and file-level explanations.

Presenter rule: explain what the evidence proves, what it suggests, and what it cannot prove. NETRA-Mail is strongest when it is transparent about uncertainty.

The 30-second explanation

NETRA-Mail is an explainable, multilingual email-threat analysis and digital-forensics platform. A user selects an open Gmail message, receives a risk score and clear reasons, and can open the exact investigation in a SOC dashboard. The platform examines sender identity, authentication evidence, language and social-engineering cues, URLs, domains, attachments, QR codes, OCR text, observed mail-server hops and relationships to earlier messages. It then preserves evidence, supports cases and campaigns, and generates reports.

The one-line pitch

NETRA-Mail turns one suspicious email into an explainable, investigation-ready and evidence-preserving security case.

1. The problem NETRA-Mail solves

Email attacks are difficult because no single clue is always decisive. A malicious email may use a normal-looking sender name, a new domain, an urgent payment request, a QR code, a misleading link, mixed languages and an apparently valid HTTPS certificate. A legitimate message can also contain words such as “payment”, “verification” or “login”. A good system therefore needs evidence fusion, context and careful handling of uncertainty.

Traditional filters often provide only a label. NETRA-Mail connects four activities that are usually separated:

1. **Detection** — find suspicious behavior and technical indicators.
2. **Explanation** — show exactly which observations contributed to the decision.
3. **Investigation** — connect URLs, domains, IP infrastructure and related emails.
4. **Forensics** — preserve hashes, evidence versions, custody events and reports.

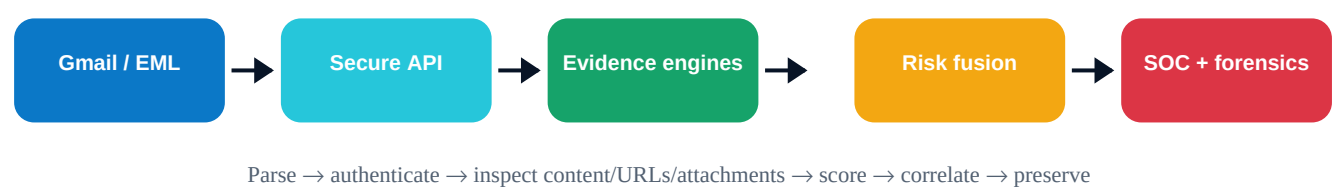
Who uses it?

User	What NETRA-Mail gives them
Gmail user	A user-triggered risk badge, score, short explanation and report link
SOC analyst	Findings, authentication evidence, URL/domain context, origin trace and relationships
Incident responder	Cases, notes, tags, evidence, custody events and reports
Security evaluator	Repeatable tests, explicit limitations and measurable model results

Why multilingual analysis matters

Phishing is not limited to English. Attackers use native scripts, Romanized regional languages and mixed-language sentences to reach more victims and avoid English-only rules. NETRA-Mail currently contains security-cue profiles for English, Hindi, Tamil, Telugu, Kannada and Malayalam, including selected native-script and Romanized phrases. Language is context, not proof of malicious intent.

User-triggered analysis to investigation-ready evidence



2. Email security from the beginning

2.1 What an email contains

An email has more than the text visible in Gmail:

- **Envelope information** is used by mail servers during delivery. The envelope sender can differ from the visible From address.
- **Headers** describe fields such as From, Reply-To, Return-Path, Message-ID, Date, Received hops, DKIM signatures and reported authentication results.
- **Body parts** may contain plain text and HTML versions.
- **Attachments** are MIME parts with filenames, declared content types and raw bytes.
- **Links and embedded resources** can appear in visible text, HTML attributes, images, QR codes or documents.

The Gmail page exposes only part of this evidence. An original .eml file is richer because it contains the original MIME structure, raw headers and attachment bytes.

2.2 What phishing means

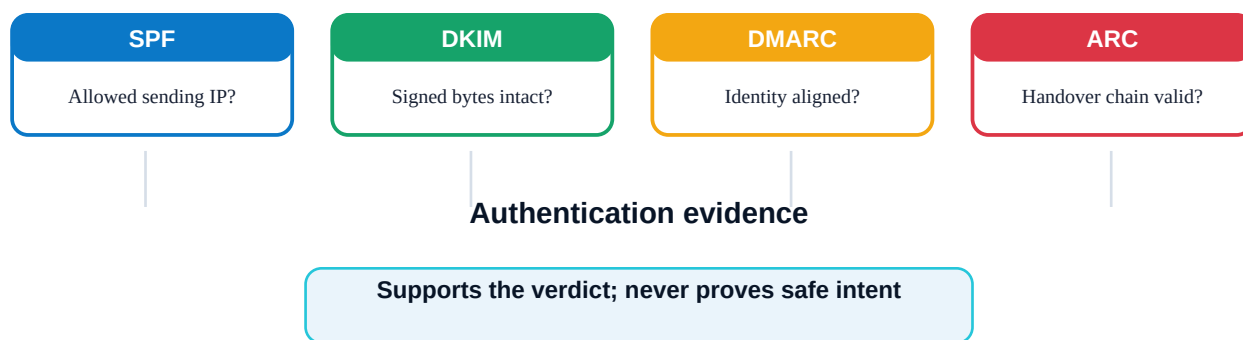
Phishing is an attempt to manipulate a user into revealing information, opening harmful content, sending money or performing an unsafe action. Common forms include:

- **Credential phishing:** fake login or verification page.
- **Business Email Compromise (BEC):** payment, invoice or bank-detail manipulation.
- **Malware delivery:** dangerous attachment or download.
- **QR phishing:** a QR code hides the destination from normal link inspection.
- **Brand impersonation:** a lookalike domain or misleading display name.
- **Conversation hijacking:** malicious content inserted into a trusted-looking thread.

2.3 The difference between identity and intent

Authentication can show that a domain authorized a sending system or signed a message. It cannot prove that the sender's account was not compromised, that the domain is honest, or that the message is safe. NETRA-Mail therefore treats authentication as one evidence family rather than a final verdict.

3. SPF, DKIM, DMARC and ARC in plain language



3.1 SPF — who is allowed to send for the envelope domain?

SPF means Sender Policy Framework. A domain publishes a DNS rule listing systems allowed to send mail for that domain. The receiving server compares the connecting IP address with that rule.

Think of SPF as a guest list checked at the building entrance. It answers: "Was this delivery server allowed to send for the envelope-sender domain?" It does not directly authenticate the visible From address and normally must be evaluated during SMTP receipt. A copied Gmail page does not contain the trusted connection context needed to recompute SPF.

3.2 DKIM — was the signed content changed?

DKIM means DomainKeys Identified Mail. The sender adds a cryptographic signature covering selected headers and body content. The verifier obtains the public key from DNS and checks the signature.

Think of DKIM as a tamper-evident seal. It answers: "Do the signed bytes still match, and which domain signed them?" Verification requires the original signed message bytes. A reconstructed email may look identical to a human but cannot be treated as the signed original.

3.3 DMARC — does authentication align with the visible From domain?

DMARC means Domain-based Message Authentication, Reporting and Conformance. It checks whether a passing SPF identity or DKIM signing domain aligns with the domain shown in the visible From address. It also lets a domain publish handling and reporting policy.

Think of DMARC as comparing the name on the letter with the identity that passed the entrance check or signed the seal. DMARC can pass through aligned SPF or aligned DKIM. A pass reduces some spoofing risk but does not prove that the message content is safe.

3.4 ARC — what happened through forwarding intermediaries?

ARC means Authenticated Received Chain. Forwarders can preserve a signed chain of authentication observations as mail travels through intermediaries. ARC helps a receiver reason about authentication that may have been changed by forwarding.

Think of ARC as a chain of signed handover receipts. A valid chain supports custody continuity; it does not certify the message as harmless.

3.5 How NETRA-Mail treats unavailable evidence

NETRA-Mail uses three important states: **pass**, **fail** and **unavailable**. Missing original bytes, missing trusted SMTP context or missing DNS evidence is reported as unavailable. It is not silently converted into pass or fail.

Mechanism	Requires	What it can support	What it cannot prove
SPF	Trusted receiving IP and DNS policy	Envelope-domain sending authorization	Visible sender identity or safe intent
DKIM	Original signed bytes and DNS key	Signed-content integrity and signing domain	That the signer is trustworthy
DMARC	Aligned SPF or DKIM evidence	Protection against some From-domain spoofing	That an authenticated account is uncompromised
ARC	Original ARC chain and signatures	Authentication custody through intermediaries	Safety, ownership or physical attribution

4. End-to-end system workflow

Step 1 — user chooses an email

The Chrome Manifest V3 extension runs inside Gmail. Analysis is user-triggered through **Analyze current email**. The extension extracts the available subject, sender, recipients, visible body, HTML and accessible headers. It sends only that selected message to the configured HTTPS backend.

Step 2 — backend validates the request

FastAPI enforces request limits, origin policy, role-based API identities and hosted deployment safeguards. The Gmail extension can submit through its pinned extension origin, while SOC and administrative routes remain authenticated.

Step 3 — parsing and normalization

The parser converts input into a stable internal representation: metadata, text bodies, HTML references, attachments, hashes, headers and extracted indicators. It avoids rendering untrusted HTML and never executes an attachment.

Step 4 — independent evidence engines run

- Sender and header consistency
- SPF/DKIM/DMARC/ARC evidence
- English and multilingual security cues
- HTML and social-engineering patterns
- URL structure, domain and optional reputation

- Bounded email and URL ML inference
- Attachments, archives, QR codes and OCR text
- Received-hop and public-infrastructure analysis

Step 5 — evidence is fused

The orchestration and risk layers combine corroborating findings into a normalized 0–100 score, classification and confidence. A model prediction is bounded so it cannot create a verdict without explainable independent evidence.

Step 6 — results are connected to investigation

The Gmail badge shows the score. A scoped link opens the matching investigation in the Streamlit SOC dashboard. The record can be correlated, added to a case, preserved as evidence and included in a report.

5. Understanding the score and decision

5.1 Score, classification and confidence are different

- **Risk score** represents accumulated evidence strength on a 0–100 scale.
- **Classification** describes the most defensible interpretation, such as legitimate/low risk, phishing, credential phishing, BEC or malware delivery.
- **Confidence** describes how complete and mutually supporting the available evidence is.

A message can have a low score and limited confidence when important headers are unavailable. A high confidence value does not mean mathematical certainty.

5.2 Why evidence correlation matters

The system avoids making strong decisions from weak single words. “Payment” in a booking confirmation is normal. “Payment” plus a verified bank-change instruction plus impersonation evidence is much stronger. Similarly, a model score becomes decision-relevant only when structural or contextual evidence independently supports it.

5.3 The IRCTC false-positive lesson

A legitimate IRCTC ticket confirmation was initially scored 95/100 because routine words, trailing text after a visible URL and Gmail redirect wrappers were interpreted incorrectly. The general parsing and context rules were corrected rather than allowlisting one sender. The same preserved message then scored 10/100 and was classified as legitimate or low risk. This demonstrates why realistic false-positive review is part of engineering, not an afterthought.

6. Detection engines and their contribution

6.1 Sender and header analysis

The system compares From, Reply-To, Return-Path, Message-ID domains and display names. It parses Received hops when original headers are present. A mismatch is evidence for review, not automatic proof of phishing, because legitimate services and mailing systems can use different domains.

6.2 Multilingual and NLP analysis

The content engines look for contextual combinations involving urgency, credentials, OTP requests, financial manipulation, secrecy and authority. They identify native-script, Romanized and mixed-language evidence. Ordinary business language is filtered to reduce false positives.

6.3 URL analysis

Static URL features include hostname structure, punycode, mixed scripts, raw IP hosts, unusual ports, embedded credentials, encoding, redirect wrappers, suspicious TLD context, subdomain patterns and visible-link mismatch. Known Gmail/Google redirect wrappers are unwrapped so the actual destination is analyzed.

For a domain absent from the training dataset, the heuristic engine still works because it inspects the URL itself. The character-based URL model also generalizes from patterns, but NETRA-Mail requires independent structural evidence before ML can strengthen the message score.

6.4 Domain, IP and reputation intelligence

DNS, registration context and optional maintained-provider reputation can add evidence. Provider data is best-effort and may be unavailable. Remote reputation is opt-in because sending a URL to a provider discloses that URL.

6.5 Attachments, archives, QR and OCR

The attachment analyzer uses filenames, extensions, MIME declarations, magic bytes, hashes and archive-member metadata. Image processing can decode QR destinations and extract bounded OCR text. A constrained Docker worker applies resource limits. NETRA-Mail performs static inspection and does not execute malware.

6.6 Email ML and URL ML

The project uses ML as supporting evidence rather than an opaque final authority. The evaluated URL model is connected to runtime inference through a bounded adapter. Email ML is deliberately limited because its current labelled corpus is small. This design favors explainability and lower false-positive risk.

6.7 Risk fusion

The orchestrator normalizes outputs into structured findings with a rule name, severity, confidence, evidence and limitations. The risk engine combines corroborating signals, caps weak evidence and produces the final score and classification.

7. Investigation, forensics and reporting

7.1 SOC dashboard tabs

Tab	Purpose
Findings	Authentication, model availability, reasons, evidence and limitations
Origin trace	Observed mail-server hops and defensible public-origin candidates
Relationships	Shared domains, URLs, IPs and campaign hypotheses
Limitations	Missing context and boundaries that affect interpretation

If Gmail extraction does not provide a trusted Received chain, the Origin trace correctly states that evidence is unavailable. It must never invent an IP address or physical location.

7.2 Cases and campaigns

Cases organize incidents with status, severity, priority, notes, tags, linked emails, evidence and timelines. Campaign correlation groups repeated technical indicators. Shared infrastructure is an investigative hypothesis and does not prove common human ownership.

7.3 Evidence integrity

Evidence is hashed with SHA-256. The SQLite ledger chains records to help reveal later modification. Evidence objects can be versioned, linked to cases and checked for integrity. Optional AES-256-GCM storage protects evidence at rest when deployment keys are configured.

7.4 Reports

The reporting service produces JSON, HTML and PDF case reports. A report can include findings, hashes, related messages, evidence state and limitations. A hash proves byte consistency after capture; it does not prove that the source itself was truthful or guarantee legal admissibility.

8. Security, privacy and deployment

8.1 Main controls

- HTTPS-only remote origins; HTTP permitted only for localhost development.
- Manifest-pinned extension identity.
- Origin-scoped extension submission.
- SHA-256 digests for server-side API identities.
- Roles such as admin, analyst, auditor and submitter.
- Request-size, rate and archive-decompression limits.
- No attachment execution.
- Controlled URL fetching with destination validation.
- PII masking in applicable paths.
- Optional encrypted evidence storage.
- Fail-closed checks in hosted mode.

8.2 Local and hosted operation

Local development uses Uvicorn on 127.0.0.1:8000 and Streamlit on 127.0.0.1:8501. Hosted deployment uses Render with an HTTPS API, authenticated dashboard and persistent storage for the forensic ledger and evidence. Secrets belong in Render environment settings and must never be committed.

8.3 Why persistent storage matters

If a hosted service uses only an ephemeral filesystem, records can disappear during redeployment. The production-shaped configuration therefore places the SQLite ledger and evidence directory on persistent storage. Backups and recovery exercises remain necessary for real institutional use.

9. Metrics: the concepts you must understand

9.1 Confusion matrix

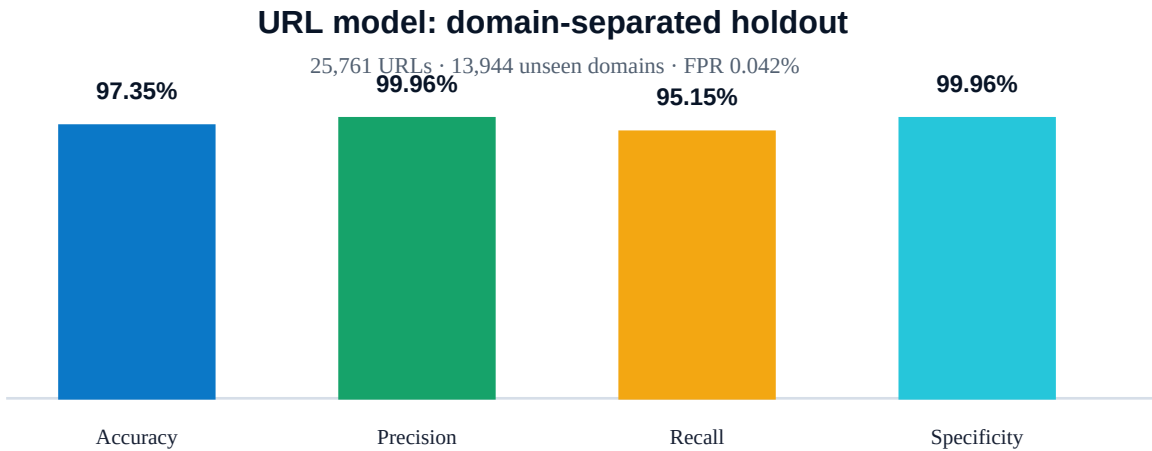
Term	Meaning
True Positive (TP)	Phishing correctly detected
True Negative (TN)	Legitimate email correctly treated as legitimate
False Positive (FP)	Legitimate email incorrectly flagged as phishing
False Negative (FN)	Phishing email incorrectly treated as legitimate

9.2 Metric formulas

- **Accuracy** = $(TP + TN) / \text{all samples}$.
- **Precision** = $TP / (TP + FP)$. Of the items flagged, how many were really phishing?
- **Recall** = $TP / (TP + FN)$. Of all phishing items, how many were found?
- **Specificity** = $TN / (TN + FP)$. Of all legitimate items, how many were left unflagged?
- **False-positive rate** = $FP / (FP + TN)$.
- **F1 score** balances precision and recall through their harmonic mean.
- **Latency** is the time required to produce a result. Warm latency excludes one-time model loading.

9.3 Why the dataset split matters

A random URL split can place URLs from the same domain in both training and test sets, making performance look better than real deployment. The stronger NETRA evaluation groups by registered domain, so all test domains are unseen during training.



10. Verified project metrics

10.1 Current automated regression suite

171 tests passed on 16 September 2026. The tests cover authentication truthfulness, DKIM verification, request limits, access control, extension security, storage encryption, ledger integrity, multilingual cues, images, archives, URL/ML integration, cases, campaigns, reporting and the legitimate IRCTC regression.

10.2 Domain-separated URL-model holdout

Measurement	Result
Training URLs	96,503
Test URLs	25,761
Unseen test domains	13,944
Accuracy	97.35%
Precision	99.96%
Recall	95.15%
F1	97.50%
Specificity	99.96%
False-positive rate	0.042%
Confusion counts	TP 13,286; TN 11,793; FP 5; FN 677

This is the strongest current model metric because domains are separated between training and test data. It measures URL classification, not complete email-system accuracy.

10.3 Seeded 25-email functional benchmark

Measurement	Result
Samples	25: 20 legitimate, 5 phishing
Novel phishing domains absent from dataset	5
Correct	25/25
False positives / false negatives	0 / 0
Warm average latency	10.83 ms
Warm P95 latency	13.35 ms
Cold start	3,412.54 ms

This benchmark demonstrates integrated behavior and runtime latency on reproducible synthetic cases. It is too small and synthetic to claim population-level production accuracy.

10.4 Honest interpretation

The project can defend strong URL-model holdout performance and broad regression coverage. It cannot yet claim that all real-world emails across every language will achieve the same accuracy. A production claim needs a large, independently labelled, time-separated email corpus with per-language and per-attack-family reporting.

11. Testing strategy

The project uses several layers of testing:

1. **Unit tests** validate individual parsers, analyzers and security controls.
2. **Integration tests** exercise analysis, persistence, evidence and reporting together.
3. **Security tests** cover authentication, origin bypass, rate/size limits and storage integrity.
4. **Corpus tests** ensure every sample produces serializable findings.
5. **False-positive regressions** preserve fixes such as the IRCTC confirmation case.
6. **Domain-separated ML evaluation** tests generalization to unseen domains.
7. **Synthetic functional benchmarking** measures latency and end-to-end behavior.

Testing files never write into real investigation storage; the test configuration uses isolated temporary paths.

12. What NETRA-Mail does well

- Connects Gmail detection directly to the matching SOC investigation.
- Explains evidence and limitations instead of returning only a label.
- Treats Indian-language and Romanized phishing cues as first-class context.
- Uses bounded ML that must be corroborated by explainable evidence.
- Supports URL, sender, attachment, QR, OCR and infrastructure analysis in one workflow.
- Preserves evidence and supports cases, campaigns and reports.
- Explicitly distinguishes unavailable evidence from pass or fail.
- Maintains responsible boundaries around IP geolocation and attribution.

13. Current limitations and future work

Present limitations

- Gmail DOM extraction cannot obtain every original header or attachment byte.

- SPF needs trusted SMTP receipt context; DKIM needs original signed bytes.
- Email-level ML training data is still too small for a production accuracy claim.
- Multilingual precision and recall require larger independent corpora per language and script style.
- Optional reputation, DNS and infrastructure providers may be unavailable.
- Static attachment inspection does not replace dynamic malware detonation.
- Correlation identifies shared indicators, not a human attacker.
- A low score is not a guarantee of safety.

Recommended roadmap

1. Trusted Gmail API or mail-gateway ingestion for original MIME and headers.
2. Larger time-separated multilingual and real-world email datasets.
3. Calibrated thresholds per attack family and language.
4. Institutional SSO, tenant isolation and role governance.
5. Durable database service, backups, monitoring and disaster recovery.
6. Optional dedicated detonation sandbox for dynamic attachment behavior.
7. Analyst feedback loop with labelled false positives and false negatives.

14. How to present NETRA-Mail to judges

A clear five-minute flow

1. **Problem:** phishing uses language, identity, URLs and attachments together; one opaque score is insufficient.
2. **Innovation:** explainable multilingual evidence fusion connected to digital forensics.
3. **Demo:** analyze one Gmail message and show the score and reasons.
4. **Investigation:** open the forensic link and show findings, authentication, relationships and evidence hash.
5. **Metrics:** lead with 171 passing tests and the domain-separated URL holdout; label the 25-email run as a functional benchmark.
6. **Honesty:** explain why missing headers are unavailable and why IP data does not identify a person.
7. **Impact:** faster user decisions, more consistent SOC triage and investigation-ready records.

Suggested closing statement

NETRA-Mail combines prevention, explanation and investigation. It helps a user recognize risk immediately, gives an analyst defensible evidence, supports Indian-language contexts and preserves the record needed to respond responsibly.

15. Common viva questions and answers

Is NETRA-Mail just an ML classifier?

No. It is a hybrid evidence system. Rules, authentication, sender consistency, multilingual cues, URL structure, ML, attachments, OCR/QR, infrastructure and forensic context contribute separately. ML is bounded and cannot create a final verdict alone.

How do you analyze a website not present in the dataset?

The static URL engine examines the URL's structure regardless of dataset membership. The character model can also generalize to patterns on unseen domains. Runtime scoring only uses ML as supporting evidence when independent structural evidence exists.

Why can a real phishing message pass SPF and DKIM?

The attacker may control the sending domain, compromise a legitimate account or use an abused service. Authentication proves specific identity/integrity properties, not harmless intent.

Why not show an attacker location for every email?

Many Gmail captures lack trusted Received headers. Even with a public relay IP, the location describes network infrastructure, not the sender's physical location. Inventing attribution would be misleading.

What reduced the IRCTC false positive?

The fix improved general parsing and context: routine request language stopped acting as executive authority, visible URL text was parsed correctly, and known Google redirect wrappers were unwrapped before analysis. No sender-only allowlist was added.

What is the most defensible metric?

The domain-separated URL holdout: 25,761 URLs across 13,944 unseen domains, 97.35% accuracy and 0.042% false-positive rate. It measures the URL model specifically. Complete email accuracy still needs a larger real-world labelled corpus.

16. Glossary

Term	Plain meaning
API	A defined way for software components to communicate
ARC	Signed authentication handover chain through mail intermediaries
ASN	Identifier for a network operator on the Internet
BEC	Business Email Compromise, usually payment or authority impersonation
CORS	Browser policy controlling which origins may call an API
DKIM	Cryptographic signature over selected email headers and body content
DMARC	Alignment and policy layer using SPF and/or DKIM
DNS	Internet directory that maps names and publishes records
EML	File containing an email's MIME bytes and headers
Evidence hash	Digest used to detect later byte changes
False negative	A malicious item incorrectly treated as legitimate
False positive	A legitimate item incorrectly flagged
IOC	Indicator of Compromise, such as a URL, domain, IP or hash
MIME	Format that structures email bodies and attachments
OCR	Optical Character Recognition; extracts text from an image
Punycode	ASCII representation used for internationalized domain names
QR phishing	A malicious destination hidden in a QR code
SOC	Security Operations Center
SPF	DNS policy authorizing sending systems for an envelope domain
SSRF	Server-Side Request Forgery; tricking a server into unsafe requests
TLD	Final domain suffix, such as .com or .in

17. Repository architecture

Folder	Contribution
backend/	FastAPI API, parsers, detection engines, intelligence, evidence, persistence and reporting
dashboard/	Streamlit SOC investigation interface and authenticated API client
extension/	Chrome Manifest V3 Gmail integration, popup, content script and service worker
data/	Trained models, metrics, datasets, predictions and safe caches
evaluation/	Reproducible evaluation runners, fixtures and portable reports
tests/	Automated correctness, security, privacy and regression tests
inspection/	Constrained Docker inspection worker definition
scripts/	Local startup, secret generation, packaging and handbook generation
docs/	Audit, deployment, release, demo and project documentation
testingforai/	Adversarial functional cases and stored challenge-suite results

18. File-by-file contribution reference

The following appendix is generated from the tracked project structure. Generated caches, virtual environments, live evidence and untracked output folders are intentionally excluded. Model files and datasets are listed, but secrets are never included.

Repository root

File	Contribution
.gitignore	Prevents caches, secrets, runtime data and generated artifacts from entering source control.
PHASE7_AUDIT.md	Project documentation or audit record; see the document title for its specific scope.
Procfile.txt	Legacy/process-platform startup declaration.
README_EVALUATION.txt	Text configuration, dependency lock or evaluation guidance.
RUN_EVALUATION.bat	Windows convenience launcher for the documented evaluation workflow.
calculate_metrics.py	Computes presentation/evaluation summaries from stored results.
dummy_phish.eml	Email fixture used for local validation or robustness testing.
evaluate_domain_holdout.py	Trains/evaluates the URL model using registered-domain-separated splits.
evaluate_netra.py	Legacy end-to-end evaluator for a running API.
phishing_evaluation_framework.md	Project documentation or audit record; see the document title for its specific scope.
project_evaluation_questions.md	Project documentation or audit record; see the document title for its specific scope.
prototype_metrics.json	Machine-readable evaluation metrics with scope and confusion counts.
pytest.ini	Configures the Python test runner.
render.yaml	Render Blueprint for the hosted API, dashboard, persistent disk and environment contract.
requirements-dashboard.txt	Minimal dependencies for the Streamlit dashboard service.
requirements-dev.txt	Development and test dependencies.
requirements-evaluation.txt	Dependencies used by evaluation utilities.
requirements-gmail.txt	Optional dependencies for trusted Gmail API ingestion.
requirements-submission.lock	Pinned dependency set for reproducible submission packaging.
requirements.txt	Primary backend runtime dependencies.
test_cases_and_examples.md	Project documentation or audit record; see the document title for its specific scope.
test_results.json	Structured configuration, manifest or stored result artifact.
test_results.txt	Text configuration, dependency lock or evaluation guidance.
test_suite.py	Python module or utility retained for implementation, compatibility or evaluation.
test_suite1.py	Python module or utility retained for implementation, compatibility or evaluation.
train_url_model.py	Builds the URL ML model and training metrics.

backend

File	Contribution
backend/.env.example	Project source, configuration or reproducibility artifact.
backend/INTEGRATION.md	Project documentation or audit record; see the document title for its specific scope.
backend/README.md	Project documentation or audit record; see the document title for its specific scope.
backend/__init__.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/access_control.py	Role-based API identities stored as token digests.
backend/app.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/attachment_analyzer.py	Bounded static attachment, archive, filename, MIME, signature and embedded-URL inspection.
backend/audit_context.py	Propagates the authenticated actor into async evidence operations.
backend/auth_analyzer.py	Interprets receiver-reported authentication without claiming signatures were locally verified.
backend/campaign_correlator.py	Scores shared-indicator relationships between messages.
backend/compliance.py	PII-preservation and privacy masking helpers.
backend/config.py	Central environment, path, limits, provider and deployment-mode configuration.
backend/controlled_fetch.py	Validated HEAD-only redirect inspection with destination-IP protections.
backend/data/training_data.csv	Labelled URL training corpus used by the URL model pipeline.
backend/database.py	SQLite forensic ledger, v2 analysis persistence, cases, campaigns, evidence and custody records.
backend/decision_engine.py	Legacy decision compatibility layer.
backend/deployment_guard.py	Fails closed when required hosted identities, storage or encryption configuration is missing.
backend/dkim_verifier.py	Verifies DKIM signatures over original bytes using bounded DNS lookups.
backend/domain_intel.py	Domain structure, DNS/registration and lookalike-domain context.
backend/email_authentication.py	Combines local DKIM/ARC verification, trusted SPF evidence and DMARC alignment.
backend/evidence.py	Creates evidence seals and hashes.
backend/evidence_service.py	Registers, versions, retrieves and verifies evidence objects.
backend/geoip_mapper.py	Backward-compatible facade over optional IP intelligence.
backend/gmail_local.py	Optional interactive Gmail OAuth connection with memory-only tokens.
backend/header_analyzer.py	Parses header identities, relay paths and inconsistency evidence.
backend/image_analyzer.py	Bounded image validation, QR decoding and portable OCR extraction.
backend/inspection_client.py	Launches one-shot resource-limited inspection containers.
backend/inspection_worker.py	Private JSON stdin/stdout entry point executed inside the inspection container.
backend/intelligence/__init__.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/intelligence/domain_provider.py	Provider-neutral domain enrichment adapter.
backend/intelligence/ip_provider.py	Provider-neutral public-IP and ASN/geolocation enrichment adapter.

File	Contribution
backend/intelligence/reputation_provider.py	Opt-in, bounded Google Safe Browsing/URLhaus-style reputation lookup.
backend/ioc_extractor.py	Extracts email addresses, domains, IPs, URLs and hashes as IOCs.
backend/llm_agent.py	Optional explanation helper; core security decisions do not depend on a remote LLM.
backend/local_extension.py	Computes and validates the portable unpacked Chrome extension identity.
backend/mailbox_ingestion.py	Fetches one explicitly selected original Gmail MIME message and submits it safely.
backend/main.py	FastAPI application, middleware, request schemas, legacy compatibility routes and versioned v2 analysis APIs.
backend/main.py.before_multilingual	Project source, configuration or reproducibility artifact.
backend/ml_classifier.py	Local email-text ML inference with availability and integrity checks.
backend/models.py	Core legacy response, evidence and decision data models.
backend/multilingual_detector.py	English and Indian-language script/Romanized security-cue detection.
backend/nlp_engine.py	Contextual phishing, urgency, credential, finance and social-engineering cue extraction.
backend/operational_status.py	Returns a credential-free inventory of enabled features and provider readiness.
backend/parser.py	Safely parses EML/MIME, headers, bodies, links and attachments without rendering hostile HTML.
backend/protected_storage.py	AES-256-GCM evidence encryption, key rotation and authenticated reads.
backend/report_service.py	Builds JSON, HTML and PDF case reports from persisted evidence.
backend/request_limits.py	Rejects oversized request bodies before parsers allocate memory.
backend/requirements.txt	Text configuration, dependency lock or evaluation guidance.
backend/schemas/__init__.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/schemas/findings.py	Structured v2 Finding, EvidenceRecord and AnalysisResult schemas.
backend/security.py	Identifier validation, constant-time key checks, rate limiting support and email masking.
backend/services/__init__.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/services/analysis_orchestrator.py	Central evidence-fusion pipeline; invokes analyzers and produces structured findings and the final analysis result.
backend/services/campaign_service.py	Persists and manages campaign relationships.
backend/services/case_service.py	Case lifecycle, email links, notes, tags and timeline operations.
backend/services/content_signals.py	Bounded static HTML and content checks without rendering or following URLs.
backend/services/investigation_graph.py	Builds graph nodes and edges for investigation visualization.
backend/services/origin_trace.py	Creates defensible origin candidates from observed public Received hops.
backend/services/risk_engine.py	Normalizes findings into score, classification and confidence.
backend/test_scoring_engine.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/test_three_components.py	Python module or utility retained for implementation, compatibility or evaluation.

File	Contribution
backend/threat_engine.py	Legacy threat-scoring compatibility engine.
backend/threat_scoring_engine.py	Explainable component-weighted scoring implementation retained for compatibility/evaluation.
backend/train_model.py	Python module or utility retained for implementation, compatibility or evaluation.
backend/url_analyzer.py	Static explainable URL engine, visible-link comparison, brand/lookalike checks and redirect-wrapper handling.
backend/url_expander.py	Routes optional URL expansion through the constrained inspection worker.
backend/url_feature_extractor.py	Produces the 29-feature schema used by the legacy URL random-forest model.
backend/url_ml_classifier.py	Bounded adapter for the evaluated URL character model.

dashboard

File	Contribution
dashboard/api_client.py	Authenticated, redirect-blocking client for dashboard-to-API requests.
dashboard/app.py	Streamlit SOC UI for overview, findings, origin trace, relationships, cases, campaigns and reports.

extension

File	Contribution
extension/background.js	Service worker that authenticates, submits selected emails and builds forensic-report links.
extension/connection.js	Shared validation for HTTPS/localhost API and dashboard origins.
extension/content.js	Extracts the open Gmail message and renders the risk badge/report link beside the subject.
extension/manifest.json	Chrome Manifest V3 permissions, pinned identity, Gmail content script and service worker declaration.
extension/popup.html	NETRA-Mail Shield popup interface and advanced connection fields.
extension/popup.js	Popup state, protection toggle, analysis command and safe connection persistence.

inspection

File	Contribution
inspection/Dockerfile	Builds the isolated, non-root, resource-constrained inspection image.
inspection/Dockerfile.dockerignore	Limits the inspection image build context.
inspection/requirements.txt	Dependencies installed only in the inspection worker image.

data

File	Contribution
data/final_feature_importance.csv	Exported URL-model feature-importance table.
data/geoip_cache/185.220.101.5.json	Deterministic cached infrastructure-enrichment response for offline tests.
data/geoip_cache/76.223.180.112.json	Deterministic cached infrastructure-enrichment response for offline tests.
data/phishing_model.joblib	Serialized local ML model artifact; loaded with schema/integrity safeguards.
data/phishtank.csv	Phishing-URL source dataset used in URL evaluation/training preparation.
data/random_forest_test_predictions.csv	Per-sample predictions retained for metric audit and error analysis.
data/training_data.csv	Labelled URL training corpus used by the URL model pipeline.
data/tranco.csv	Popular-domain reference used for legitimate URL context/evaluation.
data/url_domain_holdout_metrics.json	Machine-readable evaluation metrics with scope and confusion counts.
data/url_domain_holdout_model.joblib	Serialized local ML model artifact; loaded with schema/integrity safeguards.
data/url_domain_holdout_predictions.csv	Per-sample predictions retained for metric audit and error analysis.
data/url_features_test.csv	Test feature matrix for the legacy URL-feature model.
data/url_features_train.csv	Training feature matrix for the legacy URL-feature model.
data/url_ml_model.joblib	Serialized local ML model artifact; loaded with schema/integrity safeguards.
data/url_ml_training_metrics.json	Machine-readable evaluation metrics with scope and confusion counts.
data/url_random_forest.pkl	Serialized local ML model artifact; loaded with schema/integrity safeguards.

evaluation

File	Contribution
evaluation/deterministic/emails/01_legitimate_meeting.eml	Deterministic EML fixture for the legitimate meeting scenario.
evaluation/deterministic/emails/02_legitimate_business.eml	Deterministic EML fixture for the legitimate business scenario.
evaluation/deterministic/emails/03_credential_phishing.eml	Deterministic EML fixture for the credential phishing scenario.
evaluation/deterministic/emails/04_urgent_phishing.eml	Deterministic EML fixture for the urgent phishing scenario.
evaluation/deterministic/emails/05_bec_payment.eml	Deterministic EML fixture for the bec payment scenario.
evaluation/deterministic/emails/06_suspicious_url.eml	Deterministic EML fixture for the suspicious url scenario.
evaluation/deterministic/emails/07_ssrf_localhost.eml	Deterministic EML fixture for the ssrf localhost scenario.
evaluation/deterministic/emails/08_executable_attachment.eml	Deterministic EML fixture for the executable attachment scenario.
evaluation/deterministic/emails/09_double_extension.eml	Deterministic EML fixture for the double extension scenario.
evaluation/deterministic/emails/10_macro_document.eml	Deterministic EML fixture for the macro document scenario.
evaluation/deterministic/emails/11_html_attachment.eml	Deterministic EML fixture for the html attachment scenario.
evaluation/deterministic/emails/12_svg_attachment.eml	Deterministic EML fixture for the svg attachment scenario.
evaluation/deterministic/emails/13_suspicious_archive.eml	Deterministic EML fixture for the suspicious archive scenario.
evaluation/deterministic/emails/14_mime_mismatch.eml	Deterministic EML fixture for the mime mismatch scenario.
evaluation/deterministic/emails/15_punycode_url.eml	Deterministic EML fixture for the punycode url scenario.
evaluation/deterministic/emails/16_raw_ip_url.eml	Deterministic EML fixture for the raw ip url scenario.
evaluation/deterministic/emails/17_reply_to_mismatch.eml	Deterministic EML fixture for the reply to mismatch scenario.
evaluation/deterministic/emails/18_mixed_attack.eml	Deterministic EML fixture for the mixed attack scenario.
evaluation/deterministic/emails/19_malformed_like.eml	Deterministic EML fixture for the malformed like scenario.

File	Contribution
evaluation/deterministic/emails/20_authentication_unavailable.eml	Deterministic EML fixture for the authentication unavailable scenario.
evaluation/deterministic/results.json	Structured configuration, manifest or stored result artifact.
evaluation/deterministic/run_evaluation.py	Runs the 20 deterministic EML scenarios and records detection behavior.
evaluation/evaluate_v2.py	Runs offline v2 corpus evaluation with deterministic provider stubs.
evaluation/html_report.py	Creates portable escaped HTML evaluation reports.
evaluation/manifest_validation.py	Rejects duplicate, malformed or invalid evaluation manifests.
evaluation/random_email_benchmark.py	Generates 25 reproducible emails, including five novel phishing domains, and measures integration and latency.
evaluation/regression.py	Non-destructive final integration and regression checks.

tests

File	Contribution
tests/__init__.py	Python module or utility retained for implementation, compatibility or evaluation.
tests/conftest.py	Python module or utility retained for implementation, compatibility or evaluation.
tests/extension_security.test.cjs	Node tests for extension identity, origins, credentials, request bounds and report links.
tests/fuzz_adversarial.py	Adversarial/fuzz input runner.
tests/load_test.py	Concurrent API load utility.
tests/test_advanced_auth_image_reputation.py	Trusted SPF/DKIM/DMARC behavior, image bounds, QR/OCR and reputation privacy.
tests/test_archive_budget.py	Archive decompression-ratio and total-size protections.
tests/test_connected_workflow.py	Authenticated analysis through case, evidence and report creation.
tests/test_content_signals.py	Correlated content rules and false-positive resistance.
tests/test_corpus_pipeline.py	All corpus messages produce structured serializable findings.
tests/test_deployment_guard.py	Hosted configuration fails closed.
tests/test_detection_intelligence.py	Detection and enrichment integration.
tests/test_dkim_verification.py	Real DKIM pass, tamper failure and unavailable-DNS behavior.
tests/test_evaluation_manifest.py	Evaluation manifest quality and duplicate rejection.
tests/test_extension_submission_access.py	Pinned extension origin can submit without a shared end-user key.
tests/test_gmail_local.py	Readonly OAuth/PKCE behavior and report escaping.
tests/test_image_acceptance.py	Rotated/noisy QR, malformed images, limits and partial inspection.
tests/test_legitimate_transactional_email.py	IRCTC-style transactional-email false-positive regression.
tests/test_local_launcher.py	Portable extension ID and safe local-extension detection.
tests/test_mailbox_ingestion.py	Original MIME retrieval, size bounds, redirects and backend-origin safety.
tests/test_ml_pipeline_integration.py	Bounded URL/email ML contribution and unseen-domain behavior.
tests/test_multilingual_native_cues.py	Native-language malicious cues and legitimate-language controls.
tests/test_phase2.py	Header and infrastructure evidence.
tests/test_phase3.py	NLP detection behavior.
tests/test_phase3_origin.py	Received-hop ordering and origin-candidate logic.
tests/test_phase4.py	Privacy safeguards.
tests/test_phase4_campaigns.py	Campaign correlation and relationship persistence.
tests/test_phase5_evidence_reporting.py	Evidence lifecycle, custody and report generation.

File	Contribution
tests/test_phase6_dashboard.py	Dashboard/API contract, extension integration and unavailable-evidence UX.
tests/test_protected_storage.py	Encryption, tamper detection, key rotation and custody actors.
tests/test_request_limits.py	Pre-parser request-body capacity enforcement.
tests/test_security_harden.py	Authentication truthfulness and access-key validation.
tests/test_submitter_isolation.py	Per-submitter ownership, expiry and disabled-credential enforcement.
tests/test_system_integrity.py	Health endpoint and basic v2 input integrity.
tests/test_upgrade_regressions.py	Cross-release security and false-positive regressions.

testingforai

File	Contribution
testingforai/README.md	Project documentation or audit record; see the document title for its specific scope.
testingforai/cases/manifest.json	Structured configuration, manifest or stored result artifact.
testingforai/cases/test_1_1.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_1_2.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_1_3.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_1_4.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_1_5.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_1.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_2.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_3.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_4.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_5.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_2_6.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_3_1.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_3_2.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_3_3.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_3_4.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_4_1.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_4_2.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_4_3.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_4_4.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_5_1.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_5_2.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_5_3.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_5_4.eml	Adversarial functional EML fixture referenced by the testingforai manifest.
testingforai/cases/test_5_5.eml	Adversarial functional EML fixture referenced by the testingforai manifest.

File	Contribution
testingforai/implementation_report.md	Project documentation or audit record; see the document title for its specific scope.
testingforai/results/report.md	Stored adversarial challenge-suite result artifact.
testingforai/results/results.csv	Stored adversarial challenge-suite result artifact.
testingforai/results/results.json	Stored adversarial challenge-suite result artifact.
testingforai/run_tests.py	Python module or utility retained for implementation, compatibility or evaluation.
testingforai/test_features.py	Python module or utility retained for implementation, compatibility or evaluation.

scripts

File	Contribution
scripts/generate_project_handbook.py	Generates this Markdown handbook and styled PDF from verified project data.
scripts/generate_render_secrets.py	Generates access keys, digests and evidence-encryption material without printing private internals later.
scripts/package_submission.py	Creates a source-only, checksummed implementation-candidate archive.
scripts/start_dashboard.py	Starts Streamlit bound to localhost.
scripts/start_local.py	Starts the local API with safe defaults and operational status output.

docs

File	Contribution
docs/BASELINE_AUDIT.md	Project documentation or audit record; see the document title for its specific scope.
docs/IMPLEMENTATION_STAGES.md	Project documentation or audit record; see the document title for its specific scope.
docs/INSPECTION_AND_RELEASE.md	Project documentation or audit record; see the document title for its specific scope.
docs/NETRA_MAIL_COMPLETE_PROJECT_HANDBOOK.md	Editable source version of the complete basic-to-advanced project handbook.
docs/NETRA_MAIL_COMPLETE_PROJECT_HANDBOOK.pdf	Styled 28-page presenter handbook generated from verified project data.
docs/NETRA_MAIL_PROJECT_GUIDE.md	Project documentation or audit record; see the document title for its specific scope.
docs/RENDER_DEPLOYMENT.md	Project documentation or audit record; see the document title for its specific scope.
docs/SIH_26106_FINALIST_DEMO.md	Project documentation or audit record; see the document title for its specific scope.
docs/TEST_ACCEPTANCE_4.4.0.md	Project documentation or audit record; see the document title for its specific scope.
docs/UPGRADE_OPERATIONS.md	Project documentation or audit record; see the document title for its specific scope.

.github

File	Contribution
.github/workflows/security.yml	Project source, configuration or reproducibility artifact.

.streamlit

File	Contribution
.streamlit/config.toml	Project source, configuration or reproducibility artifact.

evaluation_results

File	Contribution
evaluation_results/email_api_results.csv	Tabular dataset or evaluation output used for reproducible analysis.
evaluation_results/metrics.json	Machine-readable evaluation metrics with scope and confusion counts.
evaluation_results/metrics_report.txt	Text configuration, dependency lock or evaluation guidance.
evaluation_results/url_results.csv	Tabular dataset or evaluation output used for reproducible analysis.
evaluation_results/v2-4.4.0.html	Project source, configuration or reproducibility artifact.
evaluation_results/v2-4.4.0.json	Structured configuration, manifest or stored result artifact.

19. Final presenter checklist

- Start with the user problem, not the technology stack.
- Explain SPF, DKIM and DMARC using identity and tamper-evidence analogies.
- Show a real Gmail analysis and open the matching forensic report.
- State that unavailable headers are not treated as passes.
- Lead metrics with the domain-separated holdout and name its scope.
- Call the 25-email run a functional benchmark, not population accuracy.
- Explain the IRCTC false-positive correction as evidence of responsible iteration.
- Close with the alert-to-investigation-to-evidence workflow.

20. Source references inside the repository

- docs/NETRA_MAIL_PROJECT_GUIDE.md — capability overview and limits.
- docs/BASELINE_AUDIT.md — staged technical audit.
- docs/RENDER_DEPLOYMENT.md — hosted deployment contract.
- data/url_domain_holdout_metrics.json — domain-separated URL results.
- evaluation/random_email_benchmark.py — reproducible 25-email benchmark.
- tests/ — automated security, correctness and regression evidence.

This handbook describes the repository at the verified revision above. Runtime providers, credentials and hosted data availability can change without changing the source code.